SSD Performance Profiling

Ethan Katz

Table of Contents

Introduction
Zero-Queue Baselines
- Mean and Median Latency
- Tail Latency at QD = 1

Block-Size and Pattern Sweep
- Throughput vs Block Size
- Latency vs Block Size

Read/Write Mix Sweep
- Throughput by Mix
- Latency and Tail Latency by Mix

Queue-Depth and Parallelism Sweep
- Throughput vs Latency
- IOPS vs Queue Depth and Little's Law

Tail-Latency Characterization
- Percentiles at Mid and Near-Knee Depth
- Latency Distribution

Working-Set and Device State
- Throughput vs Working-Set Size
- Burst vs Later Behaviour

Misleading Setup
- Buffered vs Direct Throughput
- Buffered vs Direct Latency

Conclusion
Honest AI Usage

Introduction

An NVMe SSD can handle hundreds of thousands of I/O operations per second, but only when it has many requests in flight at once. Like memory, it trades latency for throughput: adding queue depth keeps the drive busier until it hits a knee, and after that latency keeps climbing while throughput barely moves. This report measures that behaviour on my laptop's SSD and explains it in terms of queueing, the controller, interface limits, and tail latency.

The device is a Samsung PM9C1a (MZVLC1T0HFLU, 1 TB, firmware 3L2QNXH7) in a ThinkPad X1 Carbon Gen 13, attached over NVMe on a PCIe 4.0 x4 link (about 7.9 GB/s raw per direction), with the volatile write cache enabled. The host runs Ubuntu 24.04 (Linux 6.17) on an Intel Core Ultra 7 255U, and all workloads use fio 3.36 with the io_uring engine and the "none" I/O scheduler.

Since the SSD holds my root filesystem, I never benchmarked the raw device or a partition. Every workload targets a single disposable 4 GiB regular file on ext4 with direct I/O (direct=1), created once by writing every block sequentially with incompressible data so that reads reach flash instead of ext4's unwritten extents. Each run was checked first: the target had to be the expected regular file, not a symlink, exactly 4 GiB, on the expected ext4 partition, with at least 20 GiB free and the drive at 60 °C or less. Random offsets are block-aligned with a fixed seed per trial, and the data pattern is one incompressible buffer per job, held constant.

Each configuration ran three times (twice for the sustained-write test), and the runs were interleaved so any drift would hit every configuration equally. The drive temperature was sampled every second and never went above 52 °C. All 197 runs finished without a failure. Each of the seven required experiments gets its own section with two figures, followed by a conclusion.

Zero-Queue Baselines

This experiment measures latency with exactly one outstanding I/O (iodepth = 1, numjobs = 1) for the four required workloads, 30 seconds each. So QD = 1 shows what a single request costs through this software stack and device. It isn't a hardware minimum.

Workload	Mean (µs)	p50 (µs)	p95 (µs)	p99 (µs)	p99.9 (µs)	IOPS
4 KiB random read	84.8	78.3	101.9	177.2	1073	11,666
4 KiB random write	17.0	14.0	27.3	50.4	214	57,740
128 KiB	76.3	72.2	94.7	175.1	930	12,985

sequential read						
128 KiB sequential write	88.2	77.3	119.3	346.1	1335	11,241

Mean and Median Latency

Figure A compares mean latency across the four workloads. A 4 KiB random write completes in 17.0 µs on average, five times faster than a 4 KiB random read at 84.8 µs. The write is acknowledged once it reaches the drive's volatile write cache, while a random read must fetch its page from flash.

The 128 KiB sequential read takes 76.3 µs (1.70 GB/s at QD = 1) and the sequential write 88.2 µs (1.47 GB/s). The 4 KiB random read varied between trials, from 56.0 to 85.6 µs, the largest spread in the experiment. The first trial ran right after the file was created and the later ones after sequential overwrites, so device state might explain it, but I didn't isolate the cause.

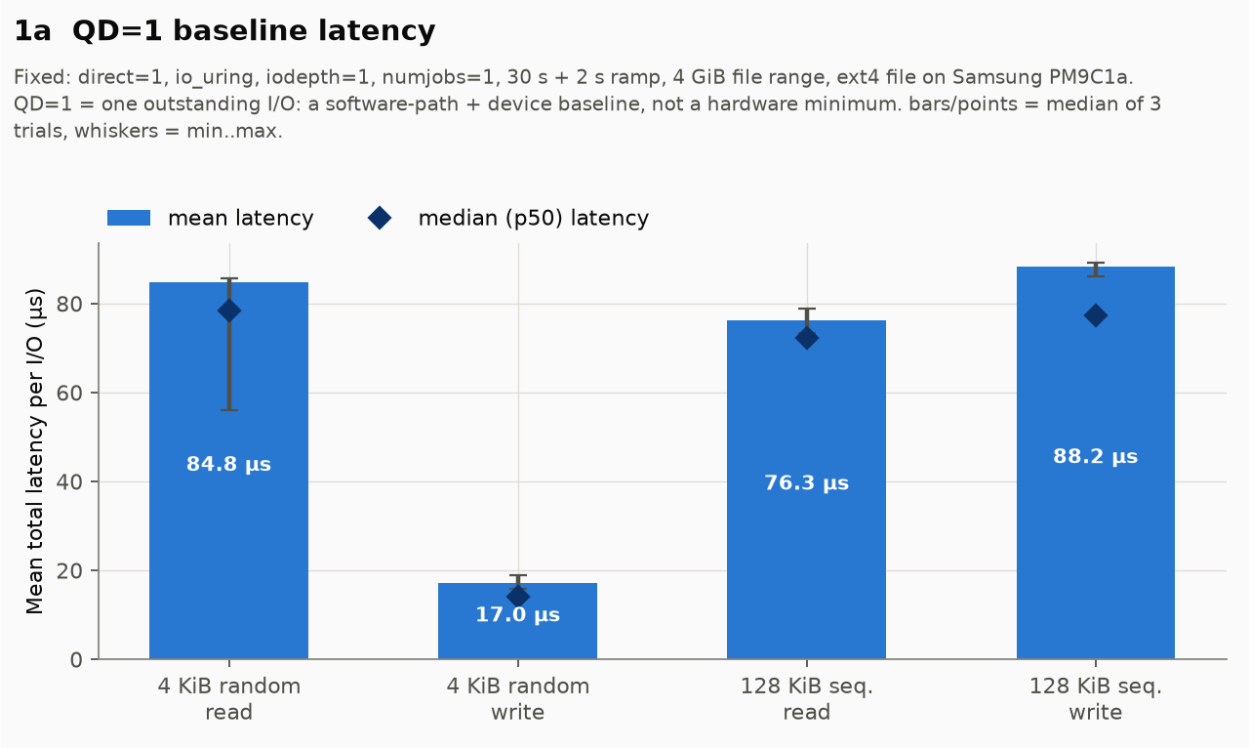


Figure A. QD = 1 mean and median latency for the four baseline workloads (median of 3 trials, whiskers show min to max).

Tail Latency at QD = 1

Figure B shows p50, p95, p99 and p99.9 on a log axis. Every workload has a long tail: p99.9 is 6 to 15 times the median, reaching 1.07 ms for random reads and 1.34 ms for sequential writes. Writes have the shortest tail at p99 (50.4 µs) but sequential writes the longest at p99.9, consistent with occasional internal work such as flushing the write cache.

Each run completed 340,000 to 1.7 million I/Os, so every p99.9 value rests on at least 340 samples. The p99.9 trial-to-trial spread was small, but it remains the least certain statistic in the table.

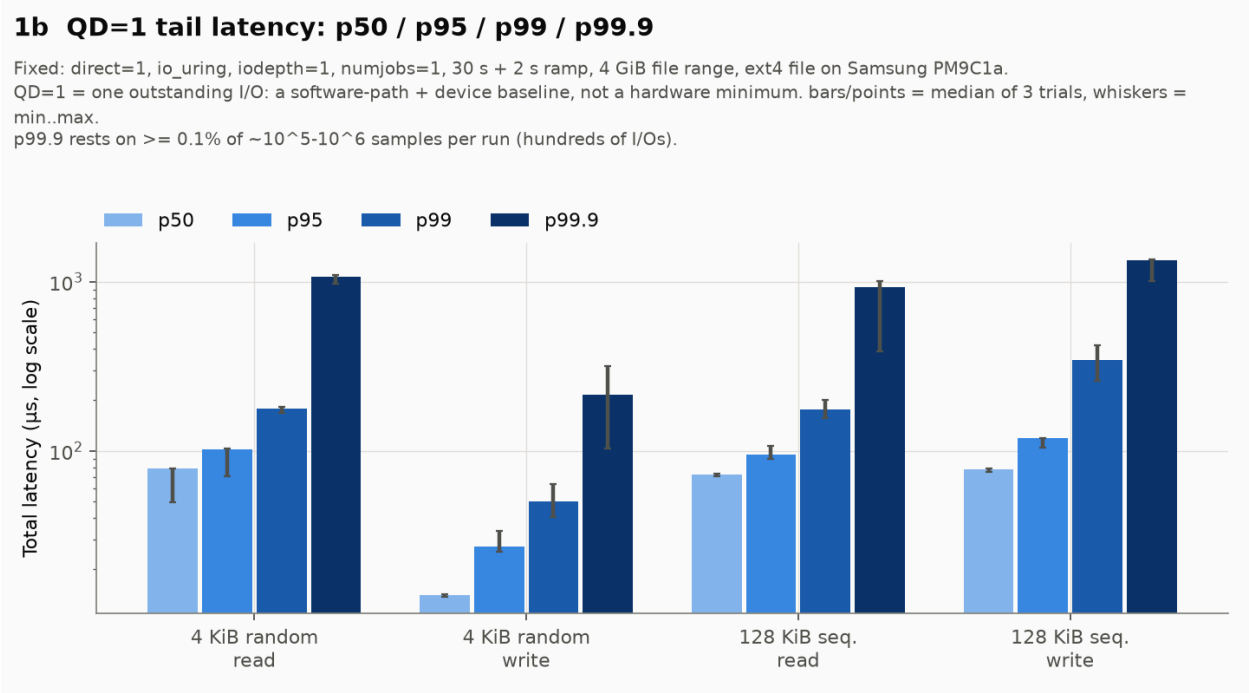


Figure B. QD = 1 latency percentiles (p50, p95, p99, p99.9) for the four workloads.

Block-Size and Pattern Sweep

This experiment sweeps block size from 4 KiB to 256 KiB for random and sequential reads at QD = 1 and QD = 32, 15 seconds per run. Throughput and latency in each pair of figures come from the same runs.

Throughput vs Block Size

Figure C shows IOPS on the left and MB/s on the right. At QD = 32, small random reads reach 345,000 IOPS at 4 KiB (1.41 GB/s), and IOPS fall as blocks grow while bandwidth rises. Bandwidth saturates near 4.6 to 4.8 GB/s from 32 KiB upward for both patterns: 4.79 GB/s for 256 KiB random reads and 4.78 GB/s sequential. Because both patterns reach the same ceiling, this is a device or controller limit, about 60% of the PCIe 4.0 x4 link's raw rate.

At QD = 1 the patterns separate. Sequential reads reach 74,300 IOPS at 4 KiB against 12,500 for random reads, and 2.60 GB/s at 256 KiB against 1.38 GB/s. So for small blocks IOPS is the number that matters, and for large blocks it's MB/s.

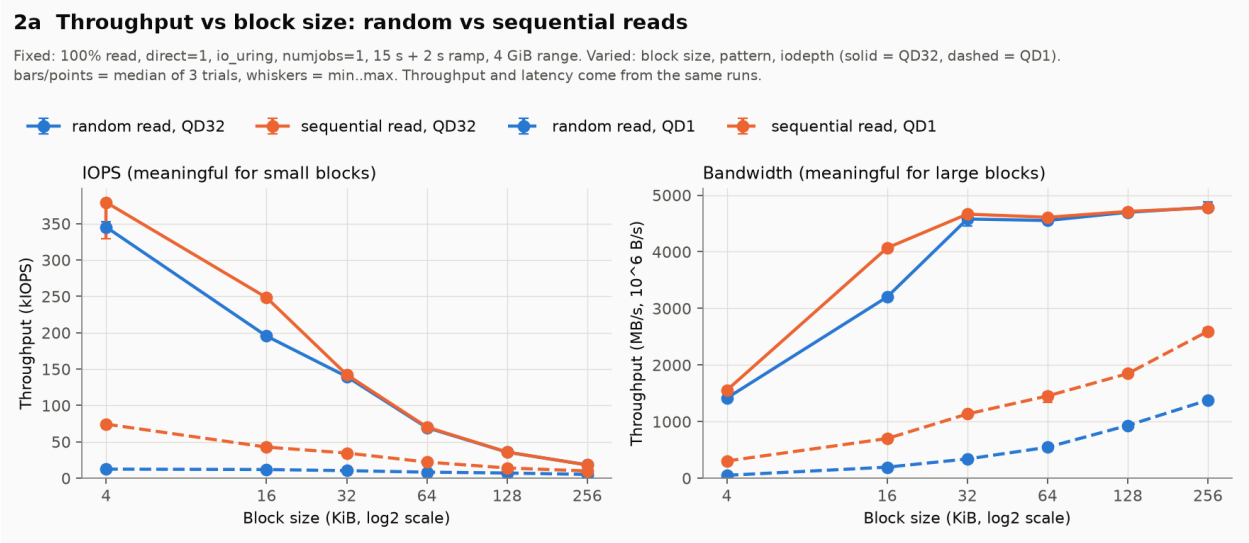


Figure C. Throughput vs. block size for random and sequential reads at QD = 1 (dashed) and QD = 32 (solid).

Latency vs Block Size

Figure D shows mean and p99 latency from the same runs. At QD = 1, sequential 4 KiB reads take 13.2 µs against 79.3 µs for random reads. The controller is likely reading ahead and serving sequential requests from its buffer, which a random pattern defeats. The gap narrows with block size, to 100.5 µs vs. 189.1 µs at 256 KiB.

At QD = 32 both patterns converge: about 85 to 93 µs at 4 KiB, rising almost in proportion to block size to about 1.75 ms at 256 KiB. Once bandwidth is saturated, a larger request simply waits longer for its share of the same 4.8 GB/s, so latency grows with size while throughput stays flat. p99 follows the same shape at roughly 1.6 to 2.7 times the mean.

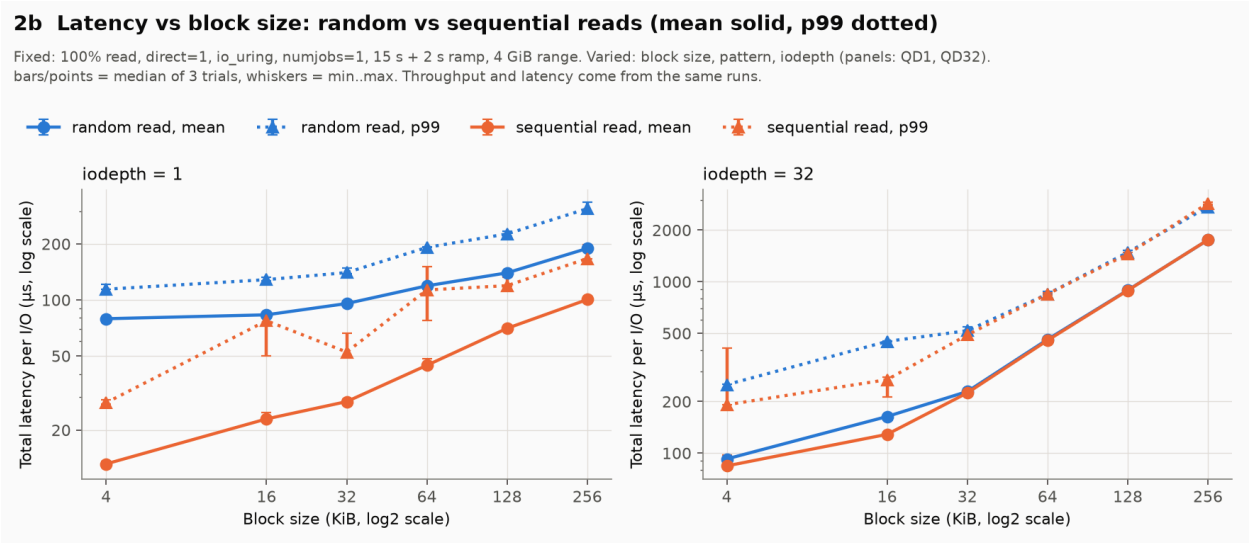


Figure D. Mean (solid) and p99 (dotted) latency vs. block size at QD = 1 and QD = 32.

Read/Write Mix Sweep

This experiment fixes 4 KiB random I/O at QD = 32 and varies the share of reads: 100%, 70%, 50% and 0% (fio randrw with rwmixread). fio reports reads and writes separately, but both directions share one queue of 32 requests.

Throughput by Mix

Figure E stacks read and write IOPS. Pure reads reach 351,000 IOPS (1.44 GB/s) and pure writes 131,000 IOPS (538 MB/s). Mixing in writes costs more than their share: a 70/30 mix delivers 220,000 IOPS in total, and 50/50 only 161,000, barely above pure writes.

Writes are slower at depth, and in a mix they also hold up the reads queued behind them. So total throughput drops fast once any writes show up.

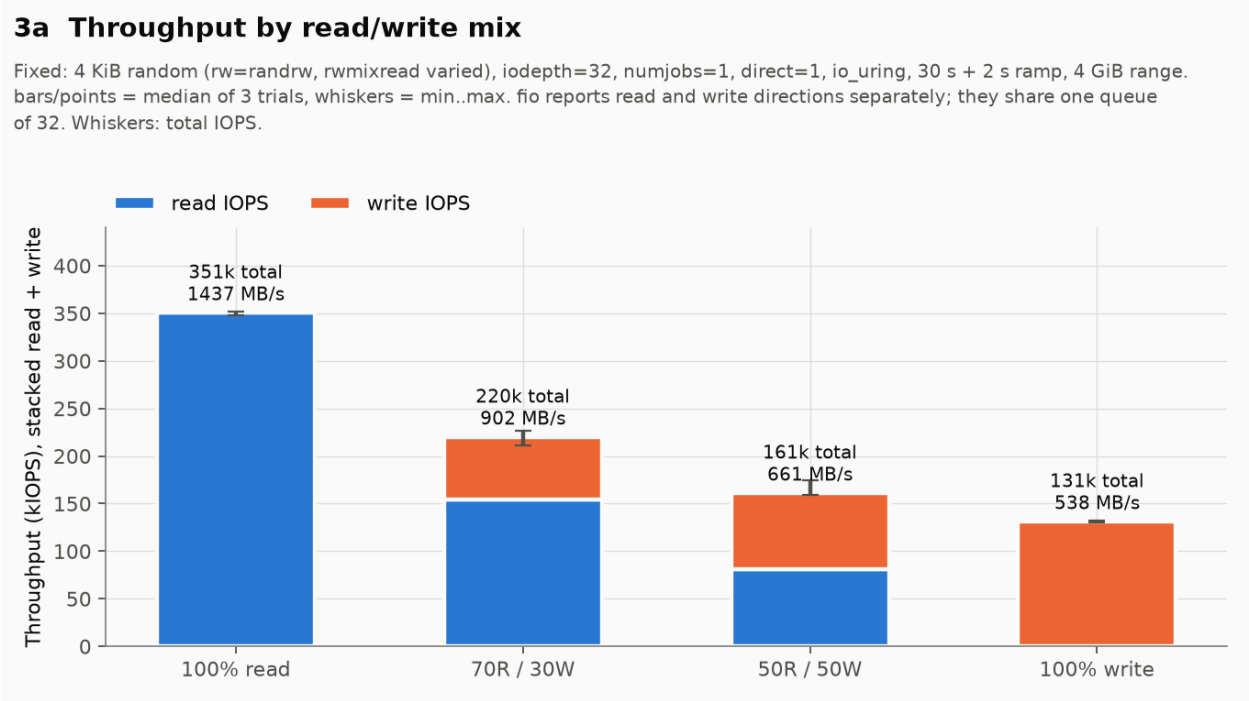


Figure E. Read and write IOPS by mix (4 KiB random, QD = 32, 3 trials).

Latency and Tail Latency by Mix

Figure F shows read and write latency for each mix. Read latency rises sharply as writes are added: mean 91 µs at 100% read, 162 µs at 70/30 and 242 µs at 50/50, with p99 climbing from 241 µs to 930 µs and 1.25 ms. Reads waiting behind writes are what stretch the tail.

Write latency grows with the write share too, from a mean of 106 µs at 70/30 to 161 µs at 50/50 and 243 µs at 100% writes, with p99 from 428 to 766 µs. These per-direction values are reported separately by fio for each mixed run.

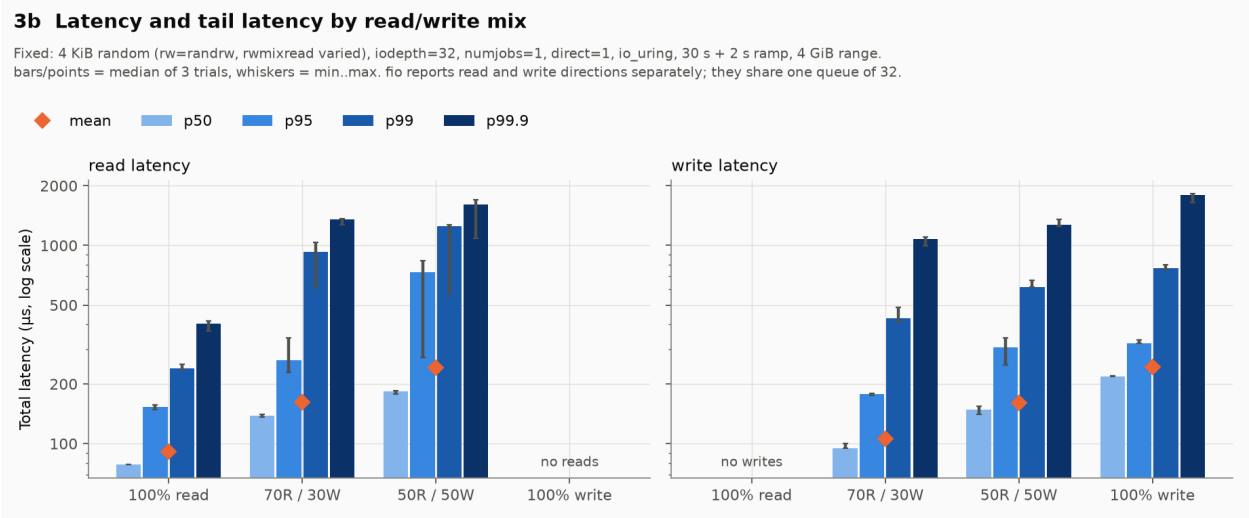


Figure F. Read and write latency percentiles by mix.

Queue-Depth and Parallelism Sweep

This experiment sweeps iodepth from 1 to 256 for 4 KiB random reads with one job (the primary curve), and separately runs four jobs with iodepth 1 to 64, aggregated with group_reporting, to test whether the plateau is a device or host limit.

Throughput vs Latency

Figure G plots IOPS against mean latency. With one job, latency stays at 80 to 83 µs from QD = 1 to QD = 16 while IOPS doubles at each step, from 12,400 to 191,700. The device serves these requests in parallel, so more depth adds throughput without adding delay.

At QD = 32 the curve bends (341,000 IOPS, 93.6 µs). Beyond it, IOPS stay at 392,000 while latency doubles with each step: 163 µs at QD = 64, 326 µs at QD = 128 and 652 µs at QD = 256. Each point is labelled with its configured depth and the achieved depth from Little's Law.

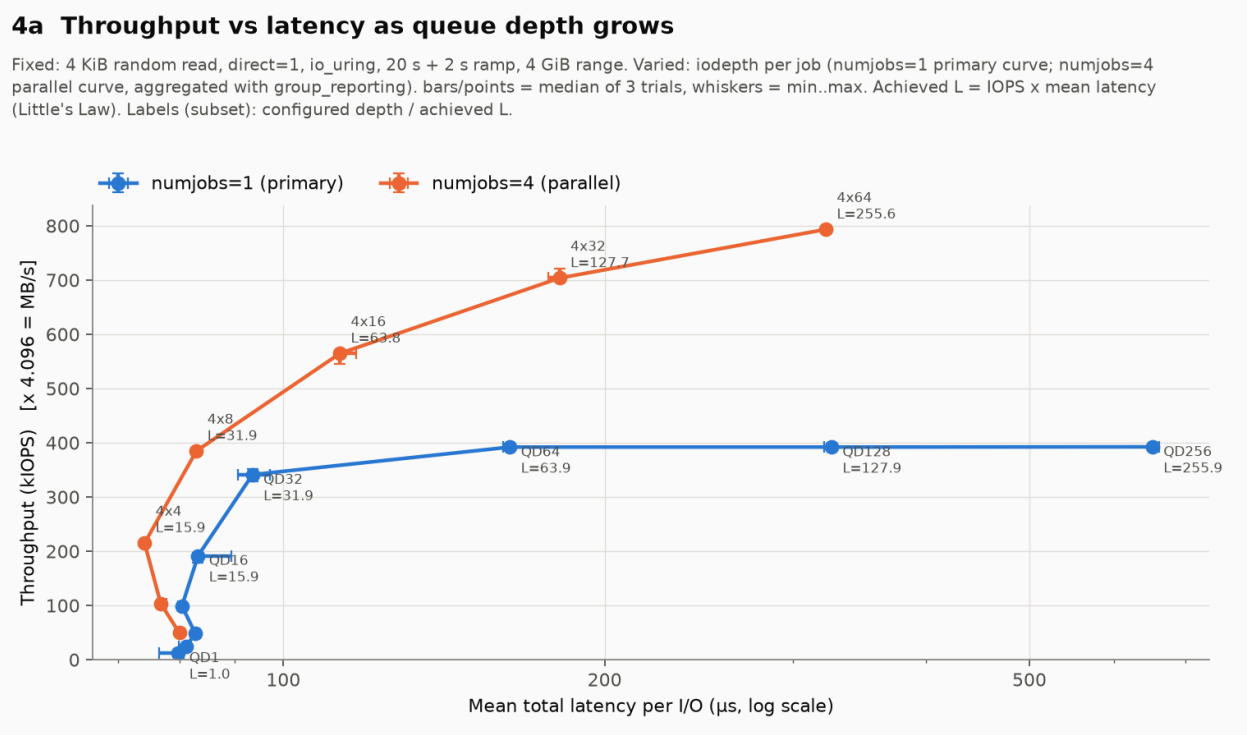


Figure G. Throughput vs. mean latency as queue depth grows, one job and four jobs.

IOPS vs Queue Depth and Little's Law

Figure H marks the knee at QD = 64, the first depth reaching 90% of peak IOPS. Little's Law says the average number of outstanding requests equals throughput times response time: 392,000 IOPS × 163 µs gives 63.9 at QD = 64, and 392,000 × 652 µs gives 255.9 at QD = 256. The achieved depth matches the configured depth at every point, so fio kept the queue full. The right panel confirms this across the whole sweep.

The one-job knee is not the drive's limit. From QD = 64 upward the single fio job uses 100% of a CPU (user plus system time), and four jobs reach 565,000, 704,000 and 793,000 IOPS at total depths of 64, 128 and 256, still rising. So the one-job plateau looks like a host-side submission limit, and the drive's own knee is somewhere past the range I tested.

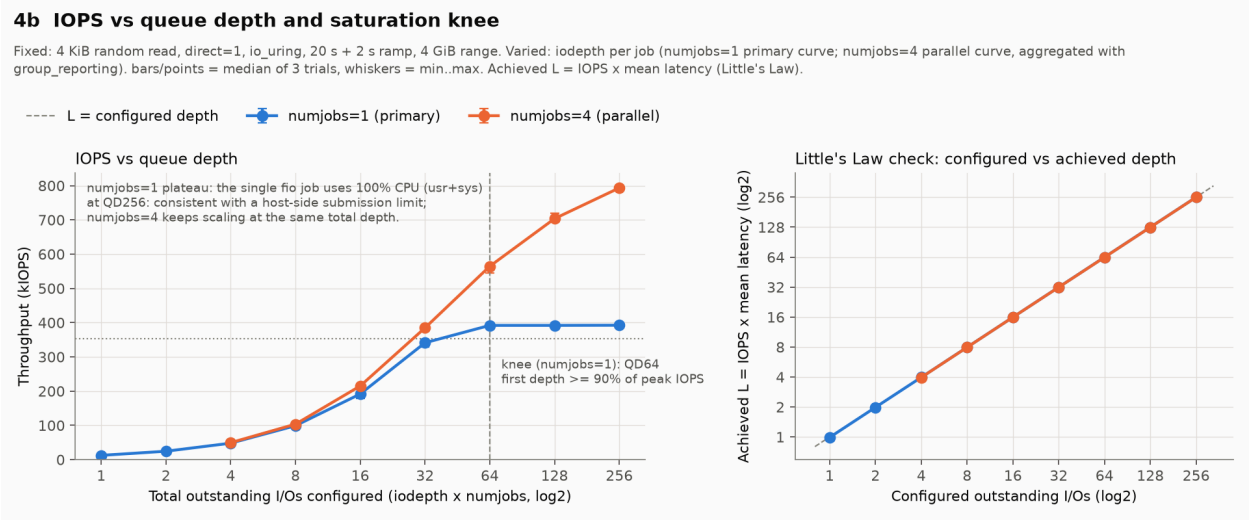


Figure H. IOPS vs. total queue depth with the knee marked (left), and achieved vs. configured depth from Little's Law (right).

Tail-Latency Characterization

This experiment compares a mid depth (QD = 16) and the near-knee depth (QD = 64) chosen from Figure H, using identical 4 KiB random reads for 60 seconds. fio's json+ output keeps the full latency histogram of every completed I/O, 11 to 23.5 million per run.

Percentiles at Mid and Near-Knee Depth

Figure I shows that moving from QD = 16 to QD = 64 roughly doubles the typical request: p50 rises from 72 to 151 µs and p95 from 142 to 224 µs. This is queueing delay, since throughput only rises from 183,000 to 392,000 IOPS. p99 grows from 204 to 293 µs and p99.9 from 412 to 668 µs.

At p99.99 both depths converge near 1.7 ms (1696 vs. 1663 µs). The extreme tail is set by occasional slow device events rather than by queue length. p99.99 rests on about 1,100 to 2,300 samples per run, enough to estimate but with visible spread between trials.

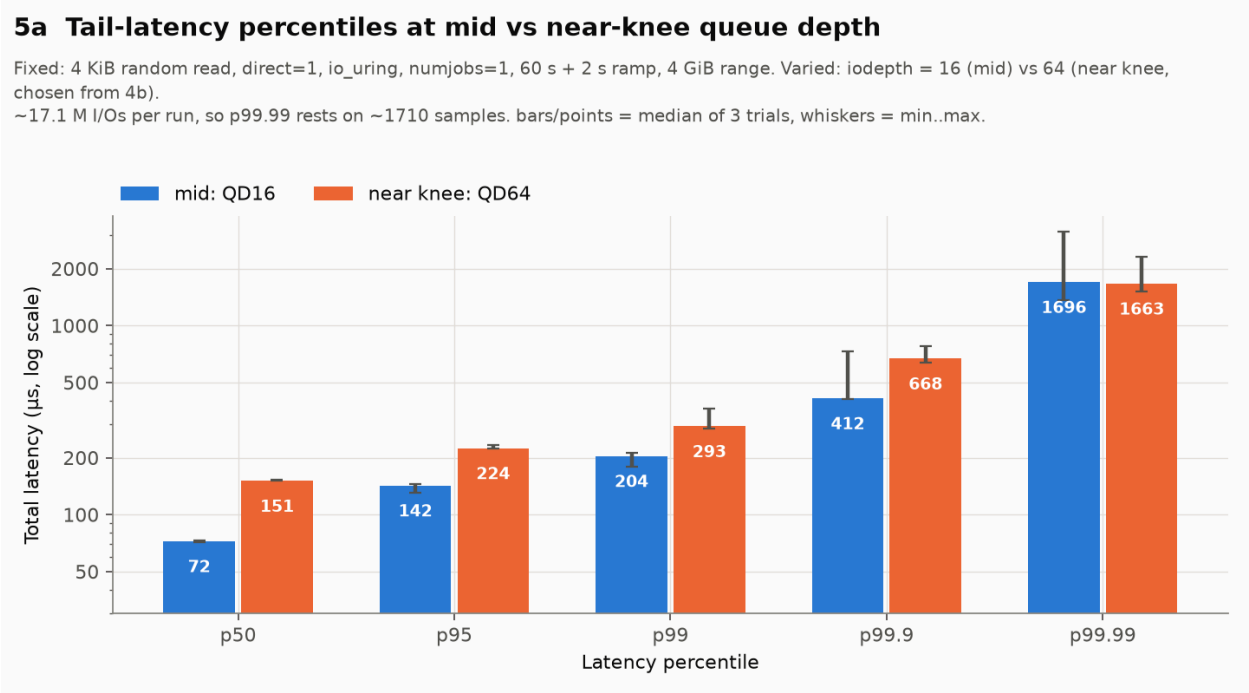


Figure I. Latency percentiles at QD = 16 and QD = 64 (4 KiB random read, 3 trials).

Latency Distribution

Figure J plots the complementary CDF from the full histograms. The QD = 64 curve is shifted to the right of QD = 16 through about the p99.9 level, the signature of queueing. Below a probability of about 10^{-4} the curves cross and overlap, and the longest requests reach 3 to 18 ms in both cases.

Trial 1 has a heavier tail than trials 2 and 3 at both depths, so the slowest requests depend partly on device state at the time of the run. Latency alone cannot identify the internal event behind them, such as garbage collection or a cache flush.

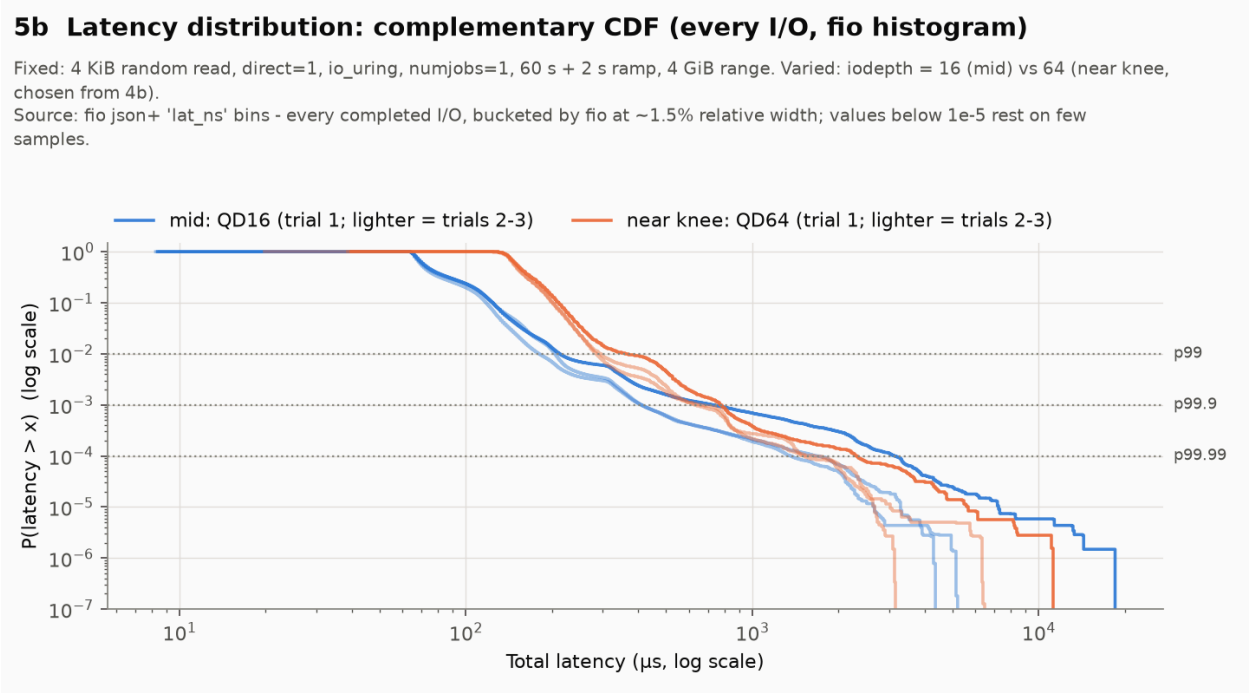


Figure J. Complementary CDF of per-I/O latency at QD = 16 and QD = 64.

Working-Set and Device State

This experiment varies the accessed range of the test file (64 MiB to 4 GiB) for 4 KiB random reads and writes at QD = 32, and runs a longer 90-second random-write test twice, with 120 seconds of idle before each, to compare an initial burst with later behaviour.

Throughput vs Working-Set Size

Figure K shows no working-set effect in this range. Random reads stay between 387,700 and 393,600 IOPS and random writes between 125,600 and 134,300 IOPS from 64 MiB to 4 GiB, with p99 latency showing no consistent trend.

All four ranges are small compared with the 1 TB drive, so the controller's address mapping and caches handle them alike. A difference would be more likely across ranges spanning tens or hundreds of gigabytes, which this bounded test avoided. The ranges are file ranges; ext4 maps them to device addresses, and their contiguity was not verified.

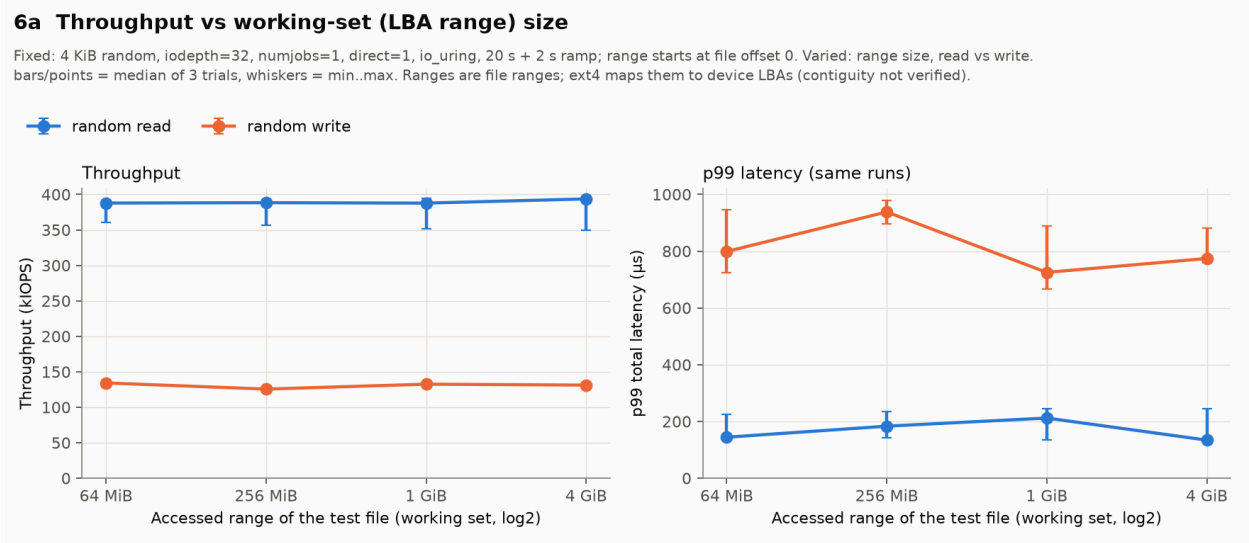


Figure K. Throughput and p99 latency vs. accessed range (4 KiB random, QD = 32).

Burst vs Later Behaviour

Figure L shows 1-second averages over the 90-second write runs. Throughput falls from 140,000 to 132,000 IOPS between the first 10 seconds and the last 30 seconds in trial 1 (6% lower) and from 142,000 to 129,000 in trial 2 (9% lower), while mean latency rises from about 228 to 246 µs. The drive warmed from 36 to 42 °C, far below its 83 °C warning threshold.

The drop repeats in both trials, so it is a real but small device-state effect. Each trial wrote about 50 GB into the same 4 GiB region, which is not enough to claim SLC-cache exhaustion or garbage-collection steady state, and the temperature rules out thermal throttling. A real steady-state test would need full-drive preconditioning, which I wasn't willing to do on my system disk.

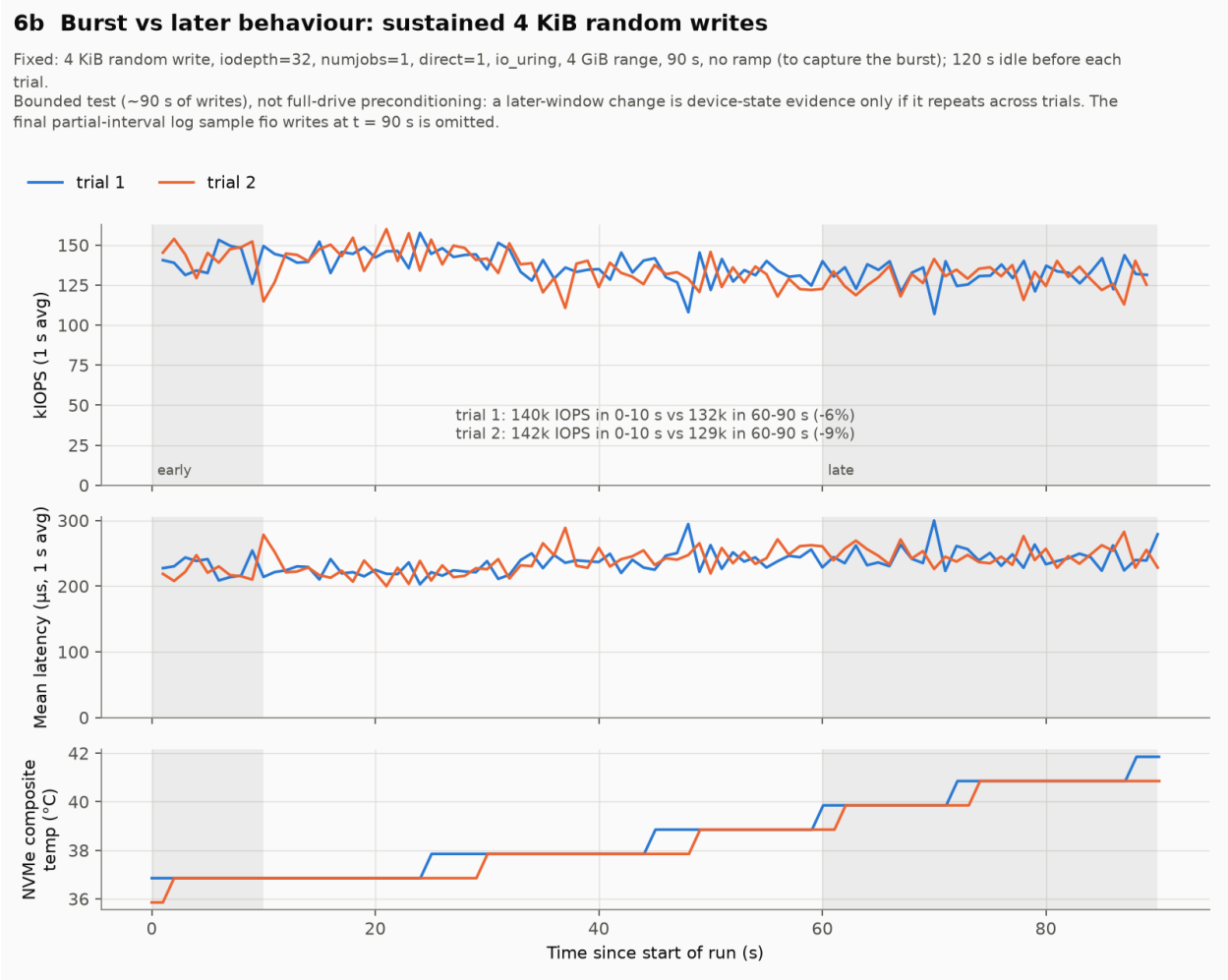


Figure L. IOPS, mean latency and drive temperature over a sustained 4 KiB random-write run (2 trials).

Misleading Setup

This experiment shows how buffered I/O can make a storage benchmark measure the OS page cache instead of the SSD. All runs use the synchronous psync engine at QD = 1 over the first 1 GiB of the test file, changing only direct=1 (device) vs. direct=0 (buffered). The cold case drops the file's cached pages first; the warm case follows a buffered pass that loads the range into memory.

Buffered vs Direct Throughput

Figure M shows direct random reads at 19,500 IOPS. Buffered reads reach 45,700 IOPS from a cold cache, because the cache fills during the run, and 268,000 IOPS from a warm cache, 13.7 times the device result. Buffered writes reach 297,000 IOPS against 67,400 for direct writes, because the write completes once the data is copied into memory and the device write happens

later.

In other words, the buffered numbers measure the page cache, not the SSD. A second misleading setup is in the data as well: psync with iodepth = 32 delivers 20,000 IOPS, the same as QD = 1, because a synchronous engine never has more than one request outstanding.

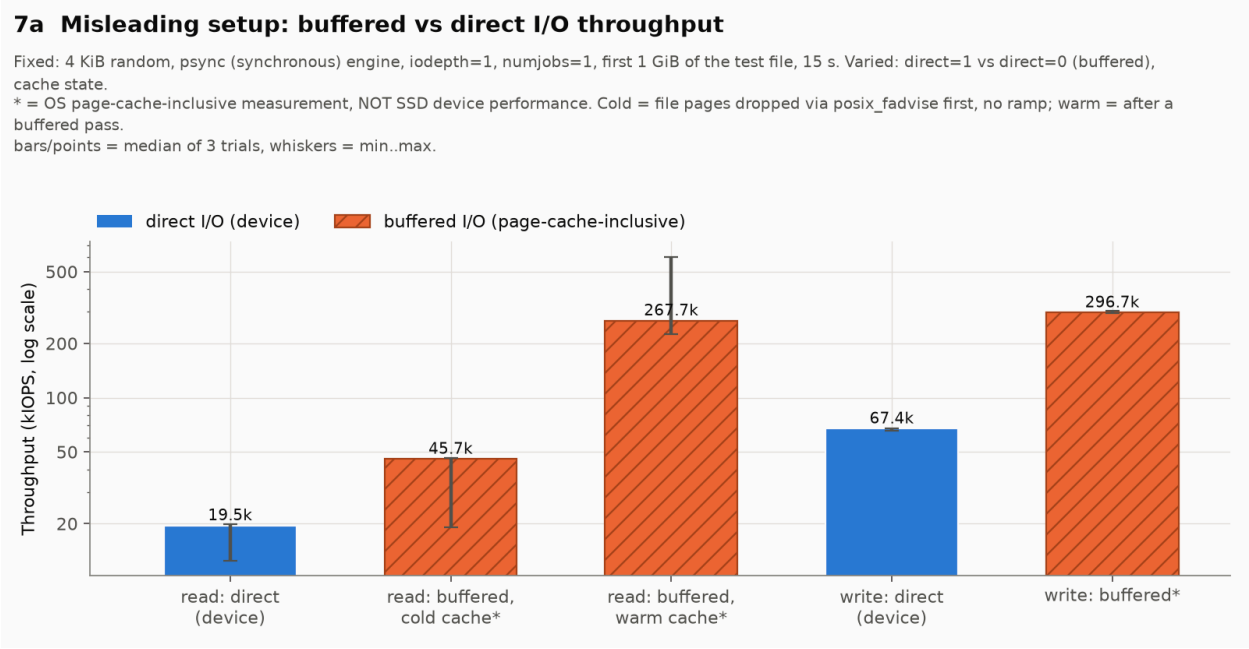


Figure M. Throughput for direct and buffered (page-cache-inclusive) I/O (4 KiB random, psync, QD = 1).

Buffered vs Direct Latency

Figure N shows the same effect in latency. Direct reads average 50.5 µs, while warm buffered reads average 3.5 µs, a memory copy rather than a flash read. Cold buffered reads average 21.4 µs, but their median is 3.5 µs while p99 is 74 µs, a mix of cache hits and device reads. Buffered writes average 3.3 µs against 14.6 µs direct.

A benchmark that omits direct=1, or reuses a file small enough to stay cached, would report latency 4 to 14 times lower than the device delivers.

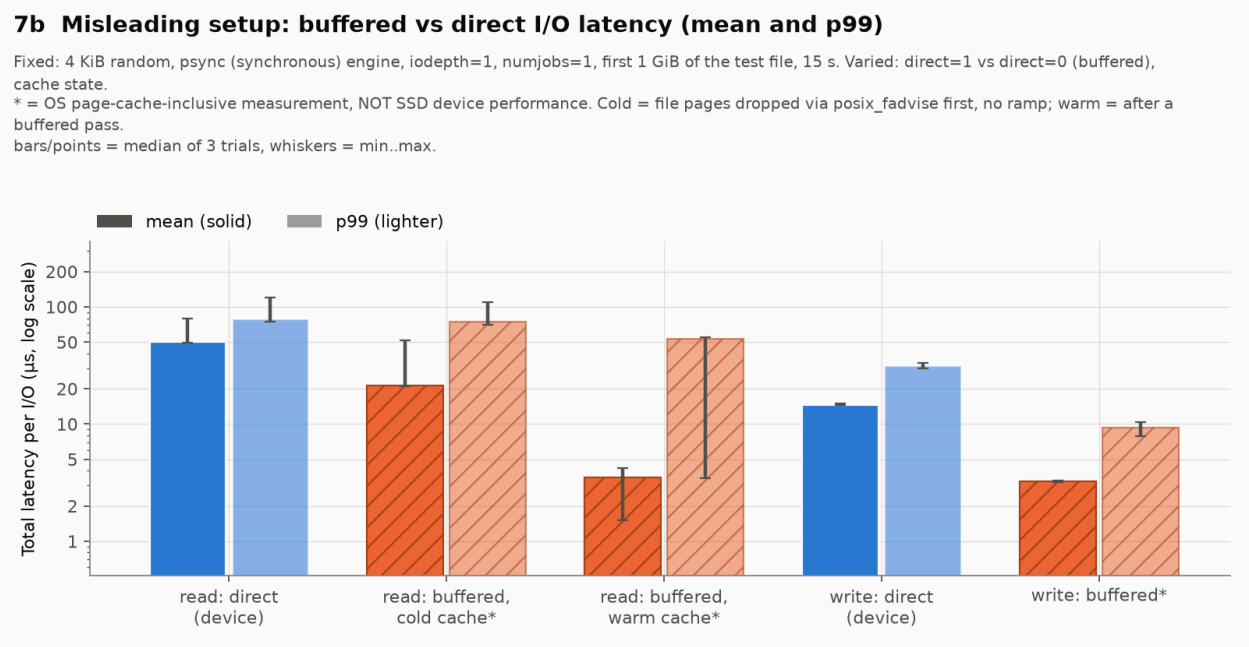


Figure N. Mean and p99 latency for direct and buffered I/O.

Conclusion

I measured a laptop NVMe SSD across queue depth, block size, read/write mix, working set, device state, and buffered vs. direct I/O. A single 4 KiB random read takes about 80 µs and a cached 4 KiB write about 17 µs, with p99.9 near 1 ms. It hides that latency through parallelism: latency stayed flat while IOPS doubled from QD = 1 to QD = 16, and one fio job saturated near 392,000 IOPS at QD = 64, where Little's Law matched the configured depth exactly and further depth only added delay. That plateau came from the host, since four jobs reached 793,000 IOPS.

Large transfers saturated near 4.8 GB/s for both random and sequential reads, and mixing in writes cut total IOPS by more than half while stretching read p99 to over 1 ms. Queueing doubled typical latency at the knee, but the extreme tail near 1.7 ms was the same at both depths. Within a 4 GiB file, working-set size made no difference, and a 90-second write burst lost only 6 to 9% of its throughput.

The main limitations are the bounded test file and write volume, which prevent a true steady-state or SLC-exhaustion measurement; the single host job behind the primary queue-depth curve; three trials per point, which makes p99.99 the least certain value; and the absence of SMART data without root access. The takeaway: how fast an SSD looks depends on how you drive it. Latency only stays low until the queue fills, and a setup mistake like buffered I/O can inflate results by an order of magnitude.

Honest AI Usage

Claude was used to help me format this report and as a supplementary tool during this project to assist with writing plotting code for CSV datasets and to provide general coding

assistance when needed. This included help with processing collected CSV data and generating visualizations for analyzing the experimental results. The AI assistance was used to support the implementation and presentation of the experiments, while the project's measurements and resulting performance analysis were based on the collected data. AI-generated code and suggestions were reviewed and adapted as needed for the project. Claude was also used to edit and condense the report's wording, label the figures, and build the table of contents; I reviewed the edited text for accuracy.